Artificial Intelligence Practicum

Emergency Response System Assignment

12 November 2026

1 Introduction

In this practicum, you will work with a large language model (LLM) to develop an AI decision-support system for emergency response after a severe storm. The city is represented as a graph, and your system must assign responders to incidents and plan routes under uncertain road conditions.

The main goal is to learn how to collaborate critically with an LLM and understand the resulting system. You should be able to explain the AI concepts used, how they are implemented in the code, and how the different components work together. This also means recognising pitfalls, questioning assumptions, and identifying where the generated solution is incorrect, incomplete, or too simple.

The assignment is deliberately open-ended. You will make your own design choices about how the system reasons, plans, and uses the available data. Different approaches are acceptable, provided that you can explain and defend your decisions and satisfy the stated requirements.

You are not expected to produce a perfect implementation. Instead, work with an LLM to create a functioning project, test its behaviour, and report honestly on what works and what still goes wrong.

2 What you will receive

File	**Description**
`city_graph.json`	Locations with x,y coordinates connected by roads with distance, speed and road types
`responders.csv`	Contains the emergency responders that must be assigned to incidents. Fields include: responder_id, type, start_location, capabilities, availability, time_until_available
`incidents.csv`	Contains the incident stream. Each incident includes: incident_id, location, priority, deadline, required_capability, timestamp, time_required
`road_reports.csv`	Noisy reports about road conditions from citizens, police, sensors.
`road_probabilities.json`	Contains the conditional probability tables (CPTs).
`observed_traffic.csv`	Contains historic traffic data on some roads in the city.
`traffic_probabilities.json`	Contains the initial, transition, and emission probability matrices.
`Scenario/Scenario`$_{\texttt{nr}}$	Contains a complete evaluation scenario with a city state, responders, incident stream, and road observations.
`city_graph.png` / `city_graph_true_pos.png`	Visual representation of the city and node locations

3 Scenario

A severe storm has affected a city, causing flooding, fallen trees, and dangerous road conditions. As part of the city's emergency coordination center, your task is to develop an AI-based decision support system that assigns emergency responders to incoming incidents and determines the routes they should follow.

The city is represented as a graph. Each node corresponds to a location, while each edge represents a road with an associated distance, speed limit, and road type. Under normal conditions, these travel times are known, but the storm introduces uncertainty about the condition of the road network.

3.1 Emergency responders

Three types of responders are available:

- **Ambulance**
  - Medical emergency
  - First aid
  - CPR
  - Life support
- **PoliceUnit**
  - Traffic control
  - Security
  - Evacuation
  - Law enforcement
- **FireTruck**
  - Fire suppression
  - Rescue
  - Evacuation
  - Pumping operations

Each responder has:

- a unique identifier;
- a start location, represented as a node in the city graph;
- a set of capabilities;
- an availability status (`Available` or `Busy`);
- if busy, the remaining time until the responder becomes available again.

3.2 Incidents

Incidents have timestamps and become known only when their timestamp is reached. Each incident contains:

- a unique identifier;
- a location;
- a priority (`Critical`, `High`, `Medium`, or `Low`);
- a required capability;
- a timestamp indicating when the incident becomes known;

- the required time to resolve the incident;
- an absolute deadline.

The required incident time represents how long the assigned responder is occupied while handling the incident after arrival.

The deadline is an absolute simulation time rather than a duration. For example, if an incident appears at simulation time 10 with a deadline of 50, the deadline is reached at simulation time 50, not 50 time units after the incident appears. The deadline refers to the latest time at which the responder must arrive at the incident location.

3.3 Uncertain road conditions

Storm conditions make road accessibility uncertain. Each road may be in one of three states:

- **Open** — normal travel time.
- **Congested** — travel time is multiplied by 1.5.
- **Blocked** — the road cannot be used.

The true road state is not directly observable. Instead, the system receives noisy reports from three sources:

- Citizens
- Police
- Sensors

In addition, the current weather condition is known and may be one of:

- Clear
- Rain
- Storm

To estimate the most likely road condition, conditional probability tables are provided in `road_probabilities.json`. These probabilities include relationships such as:

- $P(\text{Weather})$
- $P(\text{Status} \mid \text{Weather})$
- $P(\text{Reported Status} \mid \text{Status})$

3.4 Planning challenge

The objective is to assign responders to incidents while satisfying all operational constraints and minimizing response times. Keep in mind that your model does not know what will happen in the future. Incidents have a timestamp when they appear.

There is no single correct solution. Where the assignment leaves details unspecified, make reasonable assumptions, document them, and justify their consequences. Alternative interpretations and solutions are acceptable when they are consistent with the stated requirements and supported by clear reasoning and tests.

4 Required AI components

- Search
- Bayesian reasoning
- Constraint-based allocation
- Hidden Markov Models

4.1 Methods

You are required to use AI methods covered during the course. The selected methods should be applied appropriately and justified in the report.

5 Required system behavior

The final system should use the provided input data to create a feasible emergency response plan. The plan should assign responders to incidents, determine suitable routes through the city graph, and take into account responder availability, required capabilities, incident deadlines, and uncertain road conditions.

Your project must include a clearly identifiable entry point called `run.py`. Running this file for a scenario should generate two output files: `plan.json` and `statistics.txt`.

5.1 Emergency response plan

The file `plan.json` should contain a list of responder–incident assignments. Each assignment should include:

- the assigned responder;
- the incident being handled;
- the selected route through the graph;
- the simulation time at which the responder starts the assignment.

Use the following JSON structure:

```
[
    {
        "responder_id": "R1",
        "incident_id": "I1",
        "path": ["N1", "H1", "N2"],
        "time": 0
    },
    {
        "responder_id": "R2",
        "incident_id": "I2",
        "path": ["H1", "N2"],
        "time": 10
    }
]
```

The generated plan must be valid. You are responsible for defining the checks needed to establish whether routes and assignments are valid under the assignment's operational constraints. Identify possible errors, test for them, and explain any limitations of your validation.

5.2 System statistics

The file `statistics.txt` should summarize the system's performance and include at least:

- number of incidents handled;
- number of responders used;
- average response time;
- percentage of deadlines met.

Explain how you calculate these statistics, including how you treat incidents that remain unhandled. Clearly state the time units used.

An example of the expected output is:

```
System statistics:

 * 5 incidents handled
 * 2 responders used
 * Average response time: 65.7 s
 * Deadlines met: 100.0%
```

6 LLM-assisted development workflow

You will develop your system in collaboration with a Large Language Model (LLM). This collaboration is an explicit part of the assignment. You are responsible for deciding what to ask the LLM, evaluating its suggestions, and understanding the resulting implementation. The aim is to develop a working system that you can explain, test, and critically evaluate.

You must document the following:

- **LLM usage:** Which LLM tools did you use, and how did they contribute to your project? For example, describe their role in designing the system, generating code, explaining concepts, debugging, or developing tests.
- **First relevant prompt:** Include the exact text of the first substantive prompt you sent to an LLM to help design or implement this assignment.
- **Important interactions and changes:** Summarize relevant follow-up questions and answers, and explain important changes made to the generated solution. Describe which suggestions you accepted, modified, or rejected, and why.
- **Understanding:** Explain the AI concepts used, how they appear in the code, and how the components work together. You must be able to explain the submitted implementation, including code generated by the LLM.
- **Testing and findings:** Describe what you tested, how you tested it, and what you observed. Discuss what works, what fails or behaves unexpectedly, and which assumptions or simplifications affect the results.
- **Critical reflection:** Explain where the LLM was helpful and where its output was incorrect, incomplete, or misleading. Identify remaining limitations and explain what you would improve next.

A perfect implementation is not expected. A working system supported by clear explanations, meaningful tests, and an honest discussion of its limitations is the goal.

7 Defence

You will defend your submitted system in an oral discussion. The focus is on demonstrating that you understand your solution: what it does, how it works, how it applies the course concepts, and why you made particular design choices.

You should be prepared to discuss the following:

- **Your solution:** Explain how your system works and how its components work together.
- **Course concepts:** Explain how the required AI concepts are implemented in your code. Be prepared to connect the implementation to the underlying theory.
- **Design choices:** Explain and justify your choices, including assumptions, simplifications, and relevant alternatives.
- **Limitations and mistakes:** Identify concepts that are implemented incorrectly or incompletely, and design choices that cause problems. Explain their consequences and how you would address them.

- **Testing:** Explain how you tested the system, what the results demonstrate, and what remains uncertain or untested.
- **Further improvements:** Explain what you would improve next, why you would prioritize that change, and how you would check whether it helps.

You are responsible for understanding all submitted code, including code generated by an LLM. You may be asked to explain specific parts of the implementation or reason about how your system would behave in a given situation.

The defence assesses your understanding and critical judgement. Recognising a limitation and explaining a well-reasoned solution is more valuable than claiming that everything works without supporting evidence.

8 Deliverables

Submit a single PDF called `report.pdf`. Include a link to the GitHub repository containing your project, and invite both nick.wils@uantwerpen.be and lowie.rombouts@uantwerpen.be to the repository before submitting.

8.1 Deadline

The deadline of the task is 12/11/2026 at 23:59.

8.2 GitHub repository

The repository must contain all source code needed to run your system. The code should be clearly structured and documented. Include a `README.md` explaining how to install dependencies and run the project.

Your project must include an entry point called `run.py`. Running this file should generate:

- `plan.json`: the responder–incident assignments, including the selected routes and assignment start times;
- `statistics.txt`: the performance statistics specified in this assignment.

8.3 Report

Your report may contain a maximum of **3 content pages**. A title page does not count towards this limit. You may include one additional page containing relevant LLM prompts, including the first substantive prompt used to design or implement the assignment. Refer to these prompts in the main report where appropriate.

Include the GitHub repository link prominently in the report, and concisely cover:

- **Approach and implementation:** Explain your system, how the required course concepts are implemented, and the reasoning behind your main design choices and assumptions.
- **LLM collaboration:** Describe which LLM tools you used and how they contributed. Include the exact first relevant prompt used to design or implement the assignment, and summarize important follow-up interactions and changes to the generated solution.
- **Testing and results:** Describe the tests performed, including plan validation and evaluation on at least one provided scenario. Present the results and explain what you learned from them.
- **Critical reflection:** Discuss errors, incomplete components, unexpected behaviour, and remaining limitations. Explain where the LLM helped or misled you, and identify the improvements you would prioritize next.

Focus on concrete evidence and explanations that demonstrate your understanding of the submitted system.

9 Grading criteria

Criteria	**< 7**	**7–9**	**10–13**	**14–17**	**18–20**
Understanding and Defence	Cannot explain the submitted solution or the underlying AI concepts.	Explains parts of the solution, but shows major gaps in conceptual or code understanding.	Explains the main components and concepts, but struggles with implementation details, alternatives, or consequences of design choices.	Clearly connects theory to code, justifies design choices, and explains system behaviour and limitations.	Demonstrates thorough understanding of concepts and implementation. Critically evaluates alternatives, reasons through unfamiliar situations, and proposes well-motivated corrections.
LLM Collaboration	LLM usage is not documented, and the first relevant prompt is missing.	LLM usage is described superficially. Prompts, important interactions, or changes are poorly documented.	Includes the first relevant prompt and describes the LLM's contributions, with some discussion of changes and limitations.	Clearly documents relevant interactions and explains which suggestions were accepted, modified, or rejected, and why.	Demonstrates thoughtful collaboration supported by concrete examples. Critically evaluates generated code and explains how errors or misleading suggestions were identified and addressed.
System and AI Implementation	No usable system is provided, or required AI components are largely absent.	The system runs only partially. Required components contain major errors or have little connection to the system's decisions.	A working system includes the required components, although some implementations or integrations have significant limitations.	Required techniques are applied appropriately and work together coherently. Implementation choices and remaining limitations are clearly explained.	The system provides a coherent, technically sound application of the required techniques. Design choices are strongly justified, and remaining shortcomings are precisely understood.
Testing, Validation and Critical Reflection	No meaningful testing or validation is provided.	Testing is limited to a few successful runs, with little analysis of validity or failures.	Tests at least one provided scenario, reports performance statistics, and identifies basic errors or limitations.	Uses meaningful tests of components and complete plans. Analyses results, failure cases, and assumptions, and proposes justified improvements.	Provides convincing evidence through targeted tests and scenario evaluation. Clearly distinguishes demonstrated behaviour from unresolved uncertainty and explains how proposed improvements would be evaluated.

Criteria	**< 7**	**7–9**	**10–13**	**14–17**	**18–20**
Report, Code Quality and Reproducibility	The report or repository is missing, inaccessible, or insufficient to run and understand the project.	The report is unclear or incomplete. Code, documentation, or execution instructions have major gaps.	The report covers the main requirements. The code is understandable, and `run.py` generates both required output files.	The report is concise and well structured. Code is organised and documented, and the README provides clear instructions for reproducing the reported results.	The submission is clear, concise, and reproducible. The report supports its claims with evidence, and the code and documentation make the system easy to inspect, run, and explain.

10 Grading weights

Component	**Weight**
Understanding and Defence	40%
System and AI Implementation	25%
Testing, Validation and Critical Reflection	25%
Report, Code Quality and Reproducibility	10%